PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

Temas Avanzados en Ingeniería de Software 1 – INF384
PREVIO - Laboratorio 3
(Segundo Semestre 2026)

1. Objetivo

-Implementar validaciones de calidad y seguridad sobre la aplicación y publicar una imagen de contenedor, tomando en cuenta las validaciones.

-Implementar validaciones de sintaxis, integridad y seguridad sobre el código de infraestructura, validar plan y desplegar la infraestructura.

2. Pipeline de Aplicación VS Pipeline de Infraestructura

	Pipeline de aplicación	Pipeline de infraestructura
Archivo	`.github/workflows/app.yml`	`.github/workflows/infra.yml`
Se dispara con	cambios en `src/**`, `package*.json` y `Dockerfile`	cambios en `infra/**`
Es dueño de	la imagen docker que ejecuta la API (Lambda)	la existencia y la configuración de la API (Lambda)
Produce	Una imagen docker publicada en el registro	infraestructura aplicada

En el repositorio de Infraestructura, la función Lambda declara `ignore_changes = [image_uri]`, esto permite que el terraform no valide los cambios de esta URL dado que la aplicación la irá cambiando conforme libere nuevas versiones.

3. Alcance

Dentro del alcance

- Implementar buenas prácticas en el Dockerfile del repositorio base.
- Cuatro gates en el pipeline de aplicación: calidad (SonarCloud), detección de secretos, análisis de composición de dependencias y análisis estático.
- Build, tag y push de la imagen al registro, con el digest como salida del job.
- Pipeline de infraestructura completo: `fmt`, `init`, `validate`, análisis de políticas, `plan`, compuerta sobre el plan y `apply`.
- Inyectar 2 fallas para validar controles.

Fuera del alcance

- El job de despliegue. Viene escrito en el repositorio base y no se modifica.
- Escaneo de seguridad de la imagen de contenedor.

- El repositorio de imágenes (ECR) y el bucket del estado. Los crea el workflow `setup-infra` y no se declaran en Terraform.

Repositorio base: https://github.com/josedelcastillo/INF384-lab3-base

4. Prerrequisitos

- Hacer un fork del repositorio base **a su cuenta personal**.
- Sesión del entorno académico de AWS iniciada.
- Cuenta de SonarCloud configurada (pueden usar la misma del lab 2, pero el proyecto es nuevo: este es otro repositorio).
- No hay nada que instalar.

Todo el laboratorio se ejecuta en los runners de GitHub, que ya traen Docker, Terraform, la CLI de AWS y Node. No es necesario construir imágenes ni desplegar infraestructura desde su máquina, sin embargo pueden usar el GitHub Codespace si lo desean.

> El fork debe ir a una **cuenta personal**, no a una organización. Una de las actions que van a usar es gratuita solo en cuentas personales y en una organización pide una licencia.

5. Preparación previa a la sesión

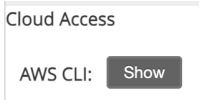
5.1 Credenciales

Las credenciales del entorno académico son temporales y caducan, deberás actualizarlas al iniciar el laboratorio.

1. Iniciar la sesión del laboratorio y copiar los tres valores: identificador de llave, llave secreta y token de sesión.
2. Cargarlos como secretos del repositorio con los nombres `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` y `AWS_SESSION_TOKEN`. (RECUERDA hacerlo como secreto, si no expones tu ambiente al mundo)
3. Ejecutar el pipeline `verificar-credenciales`. Debe terminar en verde antes de continuar.

¿Cómo obtener los valores?

- Ingresa a tu laboratorio de AWS e inícialo.
- Ingresa a “AWS Details” 

- Dar click en SHOW para que aparezcan las credenciales. 

Para validar si su token está vigente y operativo pueden ejecutar manualmente el pipeline “`verificar-credenciales`”. Tener en cuenta que las credenciales duran 4 horas apróximadamente.

Cuando algo empiece a fallar sin que hayas cambiado nada, lo primero es volver a ejecutar `verificar-credenciales`. Si sale rojo, reinicia tu laboratorio AWS y actualiza las credenciales de AWS en los secrets de Github.

5.2 Creación inicial de la infraestructura

Una función Lambda de tipo imagen exige que la imagen del docker exista en el registry (ECR de AWS) antes de crear la función. Por eso `setup-infra` deja listo todo lo que el pipeline de infraestructura necesita para poder correr:

1. El bucket de S3 donde vive el archivo de estado, con versionado, acceso público bloqueado y cifrado.
2. El repositorio de imágenes (ECR).
3. Una imagen mínima inicial, con el tag `bootstrap`.

Pasos:

1. Ejecutar manualmente el workflow `setup-infra`, pasándole como parámetro el nombre del bucket: `inf384-tfstate-<su-codigo-pucp>`.
2. Al terminar, el resumen de la ejecución imprime el contenido exacto de `infra/backend.hcl`. Copiarlo en ese archivo y **confirmar el cambio en la rama principal**.

`setup-infra` es idempotente. Si falla a mitad de camino, volver a ejecutarlo.

El archivo "`infra/backend.hcl`" se versiona: el pipeline de infraestructura lo lee en el runner y fallará si no lo encuentra.

Ese primer commit dispara el pipeline de infraestructura, creando: el grupo de logs, la política del repositorio de imágenes y la función Lambda.

Para simplificar, el lock del estado de Terraform vive en el mismo bucket y no en DynamoDB como vimos en clase. No hay que hacer nada adicional para que funcione.

6. Explicación de repositorio base

Lo que trae

- `src/`: función Node con un handler que devuelve la versión, y sus pruebas unitarias.
- `package.json` y `package-lock.json`.
- `Dockerfile`: versión con malas prácticas.
- `sonar-project.properties`: configuración del análisis de calidad, ya resuelta. No hay que editarla.
- `infra/`: código Terraform de la función Lambda y los componentes que necesita para sus logs.
- `.github/workflows/app.yml`: pipeline para validar y desplegar la aplicación. Se debe completar con los pasos requeridos, salvo el job `desplegar` que viene completo.
- `.github/workflows/infra.yml`: pipeline para validar y desplegar infraestructura. Se debe completar.
- Auxiliares que no necesitan modificación, en `.github/workflows/`:
  - `verificar-credenciales.yml`: comprueba que las credenciales de AWS estén cargadas y que la sesión siga activa.
  - `setup-infra.yml`: crea el bucket del estado, el repositorio ECR y publica la primera imagen que el Lambda necesita para poder crearse.

    - `construir-imagen.yml`: construye la imagen, reporta el tamaño de cada etapa y corre las comprobaciones del criterio de aceptación de A1, sin publicar nada.

Lo que NO trae, y hay que agregar

- `.dockerignore`. Solo viene `.dockerignore.ejemplo`, con la explicación.
- `infra/backend.hcl`. Solo viene `infra/backend.hcl.ejemplo`.

13. Errores frecuentes

Síntoma	Causa habitual	Dónde mirar
Pipeline de Setup-infra falla	La sesión del entorno académico se cerró. `sts get-caller-identity` sigue respondiendo, así que no se nota	Ejecutar `verificar-credenciales` `Actualizar credenciales en lab AWS.`
`403 Forbidden` de S3 sobre `terraform.tfstate`	Lo mismo de arriba. No es un problema del backend: sin permiso para listar, S3 responde 403 en vez de 404	Ejecutar `verificar-credenciales` `Actualizar credenciales en lab AWS.`
Error de autenticación en cualquier job de AWS	Token de sesión ausente o credenciales caducadas	Ejecutar `verificar-credenciales`
`terraform init` falla	Bucket de estado inexistente, o `backend.hcl` sin crear o mal declarado	Salida del workflow `setup-infra` y archivo `backend.hcl` en infra/
El job `calidad` falla aunque sea informativo	Falta `SONAR_TOKEN` o alguna de las dos variables. El job falla.	Secretos y variables del repositorio